

■ LEVEL 10 • QUEUE

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	First-in-first-out processing. Useful for BFS, simulations, scheduling, and level-order work.
Reference Examples	BFS • Tree Level Order Traversal • Scheduling • Simulations
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 10 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Number of Recent Calls	Queue	#933
2	Implement Queue using Stacks	Queue + Two Stacks	#232
3	Rotting Oranges	Multi-source BFS + Queue	#994
4	Sliding Window Maximum	Deque / Monotonic Queue	#239
5	Open the Lock	BFS + Queue	#752
6	Time Needed to Buy Tickets	Queue Simulation	#2073
7	Number of Recent Calls	Queue	933
8	Design Circular Queue	Circular Queue	622
9	Moving Average from Data Stream	Queue + Running Sum	346
10	Open the Lock	BFS Queue	752

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Number of Recent Calls

Pattern: Queue

LeetCode: #933

■ Input	ping times
■ Output	number of pings in the last 3000 ms
■ Constraints	times arrive in increasing order
■ Main Idea / Pattern	Push each time and pop timestamps older than t-3000.
■ Time Complexity	O(n) total
■ Space Complexity	O(n)

■ Optimal C++ Code

```
class RecentCounter {
    queue<int> q;
public:
    int ping(int t) {
        q.push(t);
        while (q.front() < t - 3000)
            q.pop();
        return q.size();
    }
};
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Implement Queue using Stacks

Pattern: Queue + Two Stacks

LeetCode: #232

■ Input	queue operations
■ Output	FIFO queue behavior
■ Constraints	standard operations
■ Main Idea / Pattern	Use an input stack for pushes and an output stack for removals; transfer only when needed.
■ Time Complexity	Amortized $O(1)$
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
class MyQueue {
    stack<int> in, out;

    void move() {
        if (out.empty()) {
            while (!in.empty()) {
                out.push(in.top());
                in.pop();
            }
        }
    }

public:
    void push(int x) {
        in.push(x);
    }

    int pop() {
        move();
        int x = out.top();
        out.pop();
        return x;
    }

    int peek() {
        move();
        return out.top();
    }

    bool empty() {
        return in.empty() && out.empty();
    }
};
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Rotting Oranges

Pattern: Multi-source BFS + Queue

LeetCode: #994

■ Input	grid of fresh/rotten oranges
■ Output	minutes until all oranges rot, or -1
■ Constraints	grid contains 0,1,2
■ Main Idea / Pattern	Put all rotten oranges in the queue first. Process the grid level by level.
■ Time Complexity	O(mn)
■ Space Complexity	O(mn)

■ Optimal C++ Code

```
int orangesRotting(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    queue<pair<int, int>> q;
    int fresh = 0;

    for (int r = 0; r < m; r++)
        for (int c = 0; c < n; c++) {
            if (grid[r][c] == 2) q.push({r, c});
            if (grid[r][c] == 1) fresh++;
        }

    int minutes = 0;
    int dr[4] = {-1, 1, 0, 0};
    int dc[4] = {0, 0, -1, 1};

    while (!q.empty() && fresh) {
        int sz = q.size();
        while (sz--) {
            auto [r, c] = q.front();
            q.pop();

            for (int d = 0; d < 4; d++) {
                int nr = r + dr[d], nc = c + dc[d];

                if (nr >= 0 && nr < m && nc >= 0 && nc < n &&
                    grid[nr][nc] == 1) {
                    grid[nr][nc] = 2;
                    fresh--;
                    q.push({nr, nc});
                }
            }
        }
        minutes++;
    }

    return fresh ? -1 : minutes;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Sliding Window Maximum

Pattern: Deque / Monotonic Queue

LeetCode: #239

■ Input	array and window size k
■ Output	maximum of each window
■ Constraints	$1 \leq k \leq n$
■ Main Idea / Pattern	Keep indices in decreasing value order. Front is always the current maximum.
■■ Time Complexity	$O(n)$
■ Space Complexity	$O(k)$

■ Optimal C++ Code

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> ans;

    for (int i = 0; i < nums.size(); i++) {
        while (!dq.empty() && dq.front() <= i - k)
            dq.pop_front();

        while (!dq.empty() && nums[dq.back()] <= nums[i])
            dq.pop_back();

        dq.push_back(i);

        if (i >= k - 1)
            ans.push_back(nums[dq.front()]);
    }
    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Open the Lock

Pattern: BFS + Queue

LeetCode: #752

■ Input	deadends and target
■ Output	minimum wheel turns to reach target
■ Constraints	4-digit lock
■ Main Idea / Pattern	Each lock state is a node. BFS explores states by number of turns.
■ Time Complexity	$O(10^4)$
■ Space Complexity	$O(10^4)$

■ Optimal C++ Code

```
int openLock(vector<string>& deadends, string target) {
    unordered_set<string> dead(deadends.begin(), deadends.end());
    if (dead.count("0000")) return -1;

    queue<string> q;
    unordered_set<string> seen;
    q.push("0000");
    seen.insert("0000");

    int steps = 0;
    while (!q.empty()) {
        int sz = q.size();
        while (sz--) {
            string cur = q.front();
            q.pop();

            if (cur == target) return steps;

            for (int i = 0; i < 4; i++) {
                for (int d : {-1, 1}) {
                    string nxt = cur;
                    nxt[i] = char((nxt[i] - '0' + d + 10) % 10 + '0');

                    if (!dead.count(nxt) && !seen.count(nxt)) {
                        seen.insert(nxt);
                        q.push(nxt);
                    }
                }
            }
        }
        steps++;
    }
    return -1;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Time Needed to Buy Tickets

Pattern: Queue Simulation

LeetCode: #2073

■ Input	ticket counts and person index
■ Output	time until a person finishes buying
■ Constraints	positive ticket counts
■ Main Idea / Pattern	Cycle through the queue. Each person can buy one ticket per turn.
■ Time Complexity	$O(\text{sum tickets})$
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
int timeRequiredToBuy(vector<int>& tickets, int k) {
    queue<int> q;
    for (int i = 0; i < tickets.size(); i++)
        q.push(i);
    int time = 0;
    while (true) {
        int i = q.front();
        q.pop();
        tickets[i]--;
        time++;
        if (i == k && tickets[i] == 0)
            return time;
        if (tickets[i] > 0)
            q.push(i);
    }
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Number of Recent Calls

Pattern: Queue

LeetCode: 933

Input	ping timestamps
Output	number of calls in the last 3000 ms
Constraints	At most 1e4 calls
Main Idea / Pattern	Keep timestamps in FIFO order and remove every timestamp older than t - 3000.
Time Complexity	O(1) amortized
Space Complexity	O(n)

Optimal C++ Code

```
class RecentCounter {
    queue<int> q;
public:
    RecentCounter() {}
    int ping(int t) {
        q.push(t);
        while (q.front() < t - 3000) {
            q.pop();
        }
        return q.size();
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Design Circular Queue

Pattern: Circular Queue

LeetCode: 622

Input	capacity and queue operations
Output	implement a fixed-size circular queue
Constraints	$1 \leq k \leq 1000$
Main Idea / Pattern	Use an array with front index and current size; wrap indices with modulo.
Time Complexity	$O(1)$
Space Complexity	$O(k)$

Optimal C++ Code

```
class MyCircularQueue {
    vector<int> q;
    int front = 0;
    int size = 0;
    int cap;
public:
    MyCircularQueue(int k) : q(k), cap(k) {}
    bool enqueue(int value) {
        if (isFull()) return false;
        int idx = (front + size) % cap;
        q[idx] = value;
        size++;
        return true;
    }
    bool dequeue() {
        if (isEmpty()) return false;
        front = (front + 1) % cap;
        size--;
        return true;
    }
    int Front() {
        return isEmpty() ? -1 : q[front];
    }
    int Rear() {
        return isEmpty() ? -1 : q[(front + size - 1) % cap];
    }
    bool isEmpty() {
        return size == 0;
    }
    bool isFull() {
        return size == cap;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Moving Average from Data Stream

Pattern: Queue + Running Sum

LeetCode: 346

Input	stream values, window size
Output	moving average of the last size values
Constraints	Window size is positive
Main Idea / Pattern	Queue stores the current window; running sum avoids recomputing it.
Time Complexity	$O(1)$ amortized
Space Complexity	$O(k)$

Optimal C++ Code

■ Absent: no exact standard LeetCode problem/code mapping was used for this extension entry.

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Open the Lock

Pattern: BFS Queue

LeetCode: 752

Input	deadends, target
Output	minimum turns to reach target
Constraints	At most 10^4 deadends
Main Idea / Pattern	Treat each lock state as a graph node and BFS by number of turns.
Time Complexity	$O(10^4)$
Space Complexity	$O(10^4)$

Optimal C++ Code

```
class Solution {
public:
    int openLock(vector<string>& deadends, string target) {
        unordered_set<string> dead(deadends.begin(), deadends.end());
        if (dead.count("0000")) return -1;
        queue<string> q;
        unordered_set<string> seen;
        q.push("0000");
        seen.insert("0000");
        int steps = 0;
        while (!q.empty()) {
            int sz = q.size();
            while (sz-- > 0) {
                string cur = q.front();
                q.pop();
                if (cur == target) return steps;
                for (int i = 0; i < 4; i++) {
                    for (int d : {-1, 1}) {
                        string nxt = cur;
                        nxt[i] = char((nxt[i] - '0' + d + 10) % 10 + '0');
                        if (!dead.count(nxt) && !seen.count(nxt)) {
                            seen.insert(nxt);
                            q.push(nxt);
                        }
                    }
                }
            }
            steps++;
        }
        return -1;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.